

LAYER 4 OF 13

AUTH PERMISSIONS

Who Gets In and What They Can Do

TIER 1 — SOLOPRENEUR

MATT MURPHY .AI

mattmurphy.ai

Part of the Builder Certification Series

What This Kit Covers

What This Kit Covers

This kit covers Layer 4: Auth & Permissions—who gets in and what they can do. Auth is short for authentication ("who are you?") and authorization ("what are you allowed to do?"). Every app that has user accounts needs both. Authentication is the lock on your front door—it verifies identity. Authorization is the set of rules about which rooms each person can enter once they're inside.

You're not going to build login systems by hand. You're going to describe what your app's access rules should be—"users can only see their own data, admins can see everything, and visitors can only see the public homepage"—and let your AI coding tool build the auth layer for you. Your job is to understand how access control works, check that the rules are enforced correctly, and make sure nobody can sneak into places they shouldn't be. That's what this certification proves you can do.

This kit walks you through three levels of mastery. Tier 1 is for someone shipping their first app with user accounts and login. Tier 2 is for someone whose app needs role-based access, team management, and secure session handling. Tier 3 is for someone running a platform with enterprise-grade security, compliance requirements, and thousands of users across multiple organizations. Start at Tier 1. You can always come back for the rest.

Why It Matters

Getting auth wrong is the most expensive mistake you can make. If your frontend looks bad, users complain. If your backend is slow, users wait. But if your auth is broken, strangers can access other people's accounts, read private data, delete records, and take actions they were never supposed to take. Auth failures don't just cause bugs—they cause security breaches, data leaks, and lost trust that you may never recover from.

Here's the thing about AI coding tools: they can set up a login page in seconds, and it will look perfectly fine. But looking fine and being secure are two completely different things. AI often skips critical details—like checking that a logged-in user can only access their own data, or properly expiring sessions, or protecting password reset flows from abuse. Auth is the one layer where "it works in the demo" is genuinely dangerous. This kit teaches you what to verify before you trust it with real users.

Certification Tier Map

There are three certification tiers. Each one builds on the one before it. Here's a quick look at who each tier is for and what you'll prove you can do.

Tier

Who It's For

What You Prove

Tier 1

Solo builders shipping their first app

You understand how login and signup work, can describe access rules to AI, and can verify that users can only see and do what they're supposed to.

Tier 2

Growing teams with role-based access needs

You can manage roles and permissions, implement team-based access, handle secure sessions, and integrate third-party login providers.

Tier 3

Platforms with compliance and multi-org security

You can oversee enterprise SSO, multi-tenant isolation, compliance-grade audit logging, and zero-trust security architecture.

TIER 1 CERTIFICATION GOAL

You can describe your app's access rules, use an AI coding tool to build login and signup, verify that users can only access their own data, and ship an app where nobody can see or do things they shouldn't.

What You Need to Know

You don't need to understand cryptography. You need to understand a handful of concepts so you can describe your access rules clearly and catch security holes in what AI gives you back.

Authentication vs. authorization (two different questions): Authentication asks "who are you?"—that's the login process. Authorization asks "what are you allowed to do?"—that's the permissions system. AI often builds authentication (the login screen works) but skips authorization (anyone who's logged in can access everything). You need both, and you need to check for both.

How login actually works behind the scenes: When a user logs in, the server checks their credentials and creates a session or a token—a digital pass that says "this person proved who they are." That pass is sent to the browser and included with every future request. If the pass is missing or expired, the server rejects the request. Think of it like a wristband at an event—you show your ID once at the door, get a wristband, and flash it everywhere else.

What a JWT token is: JWT (pronounced "jot") stands for JSON Web Token. It's the most common type of digital pass AI tools create. It's a string of text that contains the user's identity information and an expiration time. Your app sends it with every request to prove the user is logged in. You don't need to decode it—you need to know that it exists, that it expires, and that it should never be shared.

Row-level security (users can only see their own data): Just because someone is logged in doesn't mean they should see everything. Row-level security means each database row has an owner, and users can only access rows they own. For example, User A can see User A's orders but not User B's. AI almost never sets this up unless you specifically ask for it—and skipping it is a serious security hole.

The vibecoder workflow for auth: Describe your access rules clearly (who can sign up, who can log in, what each user type can see and do) → AI builds the auth layer → you test by logging in as different users and trying to access each other's data → you verify that unauthorized access is blocked → you fix any gaps → ship it. The key difference from other layers: you must test what users CANNOT do, not just what they can.

Your Toolkit

Your AI coding tool builds the auth system. These tools help you verify it's actually secure.

AI coding tool (pick one): Cursor, Lovable, Bolt, Claude Code, Windsurf, or whatever AI-assisted builder you prefer. This is where you'll describe your access rules and let AI build the auth layer.

Auth provider (Supabase Auth, Clerk, Auth0): Pre-built authentication services that handle the hard parts—password hashing, session management, email verification—so AI doesn't have to build them from scratch. Using a proven auth provider is always safer than letting AI build a custom one.

Browser incognito/private windows: The simplest way to test auth: open your app in two browser windows (one normal, one incognito), log in as two different users, and make sure each user can only see their own data. If User A can see User B's data, your auth is broken.

Certification Exam Topics

Every exam question is scenario-based. You'll see a situation and need to identify what's right, what's wrong, or what to do next. Here's what gets tested:

Authentication basics: Can you identify whether a login system properly verifies identity—checking credentials, creating sessions, and handling incorrect passwords?

Authorization rules: Can you spot when a logged-in user has access to data or actions they shouldn't—like seeing another user's account details?

Password security: Can you tell whether passwords are being stored securely (hashed, never in plain text) and whether the reset flow is protected from abuse?

Session management: Can you identify when sessions don't expire properly—leaving users logged in forever—or when session tokens are stored insecurely?

Row-level security: Can you evaluate whether your database enforces that users can only read, update, and delete their own records?

Protected routes: Can you identify when pages or API endpoints that should require login are accessible to anyone—even without being logged in?

Signup validation: Can you check that your signup flow properly validates email addresses, enforces password requirements, and prevents duplicate accounts?

Your AI workflow: Can you describe access rules to AI clearly, test the result by logging in as different users, catch security holes, and iterate to a secure result?

Common Pitfalls

These are the mistakes vibecoders make most often at this layer. No judgment—they're easy to make. But if you recognize any of them in your own workflow, fix them before sitting for the exam.

Only testing that login works—never testing what happens after login. Your login screen looks great. But did you check that User A can't access User B's data? Did you try accessing protected pages without being logged in? Auth isn't just about the door—it's about every room behind it.

Letting AI build a custom auth system instead of using a proven auth provider. AI can build a login form, but that doesn't mean it handles password hashing, token expiration, session invalidation, and brute-force protection correctly. Supabase Auth, Clerk, and Auth0 exist because auth is too important to improvise.

Storing passwords in plain text or in the database without hashing. If AI creates a users table with a column called "password" that stores readable text, that's a critical security failure. Passwords must always be hashed—scrambled into an unreadable format that can't be reversed.

Forgetting to add row-level security. Users can log in, but once they're in, they can see everyone's data. This is the most common auth gap AI creates. Always tell AI to enforce that users can only access their own records, and then verify it by testing with multiple accounts.

Sessions that never expire. AI creates a login token that lasts forever. If someone's device is stolen or their credentials are compromised, there's no way to force them out. Sessions should expire and require re-authentication after a reasonable time.

Not protecting the password reset flow. Anyone can request a password reset, the link never expires, and it can be used multiple times. A password reset link should expire quickly (15-30 minutes), work only once, and send a notification to the account owner.

Tier 1 Self-Assessment Checklist

Before you take the exam, run through these questions. Every "no" is something to work on.

Can you describe your app's access rules to AI clearly—who can sign up, who can log in, and what each user type can see and do?

Have you tested your app by logging in as two different users and confirming each can only see their own data?

Have you tried accessing protected pages and API endpoints without being logged in—and confirmed they're blocked?

Are you using a proven auth provider (Supabase Auth, Clerk, Auth0) instead of letting AI build a custom one from scratch?

Do your sessions expire after a reasonable time, and does logging out actually end the session?

Have you tested the password reset flow—checking that links expire, work only once, and notify the account owner?

Can you explain the difference between authentication (who you are) and authorization (what you're allowed to do)—and identify gaps in either one?

AI Audit Prompt Template

Copy this prompt into your AI coding tool to get a quick health check on your auth system. It checks the same things the certification exam covers.

Review my app's authentication and permissions system and check the following. For each one, tell me pass or fail with a specific example:

Authentication flow: Is the login/signup system using a proven auth provider with proper password hashing, session management, and email verification?

Authorization enforcement: Does every protected page and API endpoint check that the user is logged in AND authorized to perform the requested action?

Row-level security: Can users only access their own data—or can a logged-in user see, edit, or delete records belonging to other users?

Session management: Do sessions expire after a reasonable time, get invalidated on logout, and refresh securely?

Password reset security: Do reset links expire quickly, work only once, and notify the account owner when their password is changed?

Protected routes: Are all pages and endpoints that should require authentication actually gated—with no unprotected backdoors?

Give me an overall score out of 6 and list the top 3 things to fix first.

What's Next

Once you've gone through this kit and can answer "yes" to the self-assessment checklist, you're ready for the Layer 4 certification exam at your target tier.

The best way to prepare: build something real. Pick a project that needs user accounts—even a simple one like a personal dashboard or a shared task list—and go through the full loop. Describe your access rules to AI. Test by logging in as different users. Try to break into places you shouldn't be. Fix every gap. Ship it. Every security hole you catch during building is exactly what the exam tests.

CERTIFICATION PATHWAY

Associate Builder: Pass all 13 layers at Tier 1

Certified Builder: Pass all 13 layers at Tier 1 + Tier 2

MADE Certified: Pass all 39 tier exams + capstone project

Each exam requires 80% to pass. You can retake after a 24-hour cooldown. No rush—take the time to build something real first.

MATT MURPHY .AI

mattmurphy.ai

© 2026 Matt Murphy .AI. All rights reserved.